

Laborator 3 - Tablouri - Tehnici simple de sortare

1. Scopul și obiectivele lucrării

Scopul lucrării

Scopul acestei lucrări de laborator este de a prezenta cele mai cunoscute metode de sortare a tablourilor.

Obiectivele lucrării

Evaluarea comparativă a performanțelor unor algoritmi consacrați de sortare ce operează asupra tipului de date abstract tablou

Condiții prealabile

Pentru pregătirea acestui laborator se vor recapitula :

- Tipurile de date de baza din limbajul C
- Citirea și scrierea din/in fișiere de tip text folosind funcțiile din bibliotecile standard în C
- Declararea și parcurgerea unui tablou în limbajul C

Laborator 3 - Tablouri - Tehnici simple de sortare



3. Definiții și noțiuni teoretice

Aspecte teoretice

În cele ce urmează vom presupune că tablourile de sortat sunt definite după cum urmează:

```
typedef struct {  
    int cheie;  
    ... alte câmpuri;  
} tip_element;  
    tip_element a[nr_max_elemente];
```

Se menționează că tipul cîmpului "cheie" poate fi orice tip pe care e definită o relație de ordonare (întreg, real, sir de caractere, enumerare).

Algoritmii de sortare se deosebesc între ei prin eficiență, timp de execuție necesar, exprimat prin funcția O . Sunt folosiți pentru aprecierea eficienței și: numărul comparațiilor de chei efectuate pentru sortare (C), mai ales atunci când cheile sunt siruri lungi de caractere și numărul mișcărilor (asignărilor) de elemente (M), atunci când dimensiunea elementelor tabloului este mare, în această situație fiind indicat că pentru sortare să se folosească un tablou paralel de cursori la elementele celui inițial.

Acești indicatori depind de numărul elementelor tabloului (N).

4. Metode directe de sortare

4.1. Sortarea prin insertie

Principiu: tabloul este vazut ca fiind format din doua subtablouri $a[0], a[1], \dots, a[i-1]$ si respectiv $a[i], a[i+1], \dots, a[N-1]$ ($i=1, N-1$). Secventa $a[0], \dots, a[i-1]$ este ordonata si urmeaza ca $a[i]$ sa fie inserat in aceasta secventa la locul potrivit, astfel incit secventa $a[0], \dots, a[i-1], a[i]$ sa devina ordonata, urmind ca in pasul urmator cele doua subtablouri considerate sa fie $a[0], \dots, a[i]$ si $a[i+1], \dots, a[N-1]$.

Pentru a gasi locul in care trebuie sa fie inserat $a[i]$ se parcurge sirul $a[0], \dots, a[i-1]$ de la dreapta spre stinga, pina cind fie se gaseste un element cu cheia $\leq a[i].cheie$, fie s-a atins capatul sirului. Aici se poate utiliza metoda fanionului, extinzind tabloul spre dreapta cu elementul $a[n]$ care se asigneaza initial cu $a[i]$ (deci $TipIndex=0..N$).

Implementarea algoritmului in C:

```
void insertie(tip_element a[],int n)
{
    Tip_indice i,j;
    //fanionul pe pozitia a[n]
    for( i=n-2;i>=0;i--)
    {
        a[n]=a[i];
        j=i+1;
        while(a[j].cheie<a[n].cheie)
        {
            a[j-1]=a[j]; j++;
        }
        a[j-1]=a[n];
    }
}
```

In cazul sortarii prin insertie C si M sunt de ordinul N^2N , avind valori minime cind tabloul e ordonat si maxime cind tabloul e ordonat descrescator. Aceasta metoda este stabila.

4. Metode directe de sortare

4.2. Sortarea prin insertie binara

Principiu: reprezinta o varianta a sortarii prin insertie, in care cautarea locului de inserare se face aplicind cautarea binara, stiind ca secventa $a[0], \dots, a[i-1]$ este deja ordonata.

Implementarea algoritmului in C:

```
/*Sortare prin insertie binară - Varianta C */
void sortare_prin_insertie_binara(tip_element a[],tip_indice n)
{
    tip_indice i,j,stanga,dreapta,m; tip_element temp;
    for( i= 1; i < n; i ++ )
    {
        temp= a[i]; stanga= 0; dreapta= i-1;
        while (stanga<=dreapta)
        {
            m= (stanga+dreapta)/ 2;
            if (a[m].cheie>temp.cheie)
                dreapta= m-1;
            else
                stanga= m+1;
        } /*WHILE*/
        for( j= i-1; j >= stanga; j -- ) a[j+1]= a[j];
        a[stanga]= temp;
    } /*FOR*/
} /*SortarePrinInsertieBinara*/
```

In cadrul acestei metode, C este de ordinul $N^2 \log N$, iar M de N^2N . Se obtin valori minime ale lui C pentru tablouri ordonate invers si valori maxime pentru tablouri ordonate.

4. Metode directe de sortare

4.3. Sortarea prin selectie

Principiu: se considera subtabloul $a[i], \dots, a[N-1]$, se cauta elementul cu cheia minima din acest subtablou si apoi se interchimba acest element cu elementul $a[i]$, repetindu-se procedeul pentru valori ale lui i de la 1 la $N-2$.

Implementarea algoritmului in C:

```
void sortare_prin_selectie(tip_element a[], tip_element n)
{
    tip_indice i,j,min; tip_element temp;
    for( i= 0; i <= n-2; i ++ )
    {
        min= i; temp= a[i];
        for( j= i+1; j < n; j ++ )
            if (a[j].cheie<temp.cheie)
            {
                min= j; temp= a[j];
            } /*FOR*/
        a[min]= a[i]; a[i]= temp; /*interschimbarea*/
    } /*FOR*/
} /*SortarePrinSelectie*/
```

In cazul sortarii prin selectie C este de ordinul N^2 , iar M este de ordinul $N \ln N$. Aceasta metoda este mai putin rapida pentru tablouri ordonate sau aproape ordonate.

4. Metode directe de sortare

4.4. Sortarea prin selectie performanta

Principiu: reprezinta o varianta a sortarii prin selectie, in care determinarea elementului cu cheia minima dintr-o portiune de tablou se reduce la determinarea pozitiei acestuia. In felul acesta se poate renunta la asignarea $x:=a[j]$ care apare in ciclul "for" controlat de j.

Implementarea algoritmului in C:

```
void selectie_optimizata(tip_element a[], int n)
{
    tip_indice i,j,min; tip_element temp;
    for( i= 0; i <= n-2; i ++ )
    {
        min= i; temp = a[i];
        for( j= i+1; j < n; j ++ )
            if (a[j].cheie<a[min].cheie) min= j;
        temp= a[min]; a[min]= a[i]; a[i]= temp;
    } /*FOR*/
} /*SelectieOptimizata*/
```

4. Metode directe de sortare

4.5. Sortarea prin interschimbare (bubblesort)

Principiu: se considera subtabloul $a[i], \dots, a[N-1]$ care se parcurge de la dreapta spre stnga, comparind si interschimbind perechile de elemente alaturate care nu satisfac relatia de ordine, procedeul repetindu-se pentru $i=1, N-1$. Practic, la o parcurgere a subtabloului $a[i], \dots, a[N-1]$ are loc deplasarea elementului minim al acestui subtablou pna in pozitia $a[i-1]$.

Implementarea algoritmului in C:

```
void bubblesort(tip_element a[], int n)
{
    tip_indice i,j; tip_element temp;
    for( i= 0; i < n; i ++ )
    {
        for( j= n-1; j>i; j -- )
            if (a[j-1].cheie>a[j].cheie)
            {
                temp= a[j-1]; a[j-1]= a[j]; a[j]= temp;
            }
    }
} /*Bubblesort*/
```

4. Metode directe de sortare

4.6. Sortarea prin amestecare (shakersort)

Principiu: reprezintă o variantă a metodei bubblesort, având următoarele îmbunătățiri:

- la fiecare parcurgere a subtabloului $a[i], \dots, a[N-1]$ se memorează indicele k al ultimei interschimbări efectuate, astfel încât la următoarea trecere un capăt al subtabloului va fi marcat de k (între 1 și k tabloul este ordonat);
- se schimbă alternativ sensul de parcurgere al subtablourilor pentru două treceri consecutive.

Implementarea algoritmului în C:

```
void shakersort(int a[], int n)
{
    int j, ultim, sus, jos;
    int temp;
    sus= 1; jos= n-1; ultim= n-1;
    do {
        for( j= jos; j >= sus; j --)
        {
            if (a[j-1]>a[j])
            {
                temp= a[j-1]; a[j-1]= a[j]; a[j]= temp;
                ultim= j;
            }
        } /*FOR*/
        sus= ultim+1;
        for( j=sus; j <= jos; j ++ )
        {
            if (a[j-1]>a[j])
            {
                temp=a[j-1]; a[j-1]=a[j]; a[j]=temp;
                ultim=j;
            }
        } /*FOR*/
        jos=ultim-1;
    } while (!(sus>jos));
} /*Shakersort*/
```

La sortările bubblesort și shakersort, M și C sunt proporționale cu N^2 , metodele fiind mai puțin performante decât cele anterioare.

4. Metode directe de sortare

4.7. Sortarea prin determinarea distribuției cheilor

Problema se formulează astfel:

- Se cere să se sorteze un tablou cu n articole ale căror chei sunt cuprinse în intervalul $[0, m-1]$.

Dacă m nu este prea mare pentru rezolvarea problemei poate fi utilizat algoritmul de "determinare a distribuției cheilor" (counting sort).

Ideea algoritmului este următoarea:

- (1) Se contorizează într-o primă trecere numărul de chei pentru fiecare valoare de cheie care apare în tabloul a .
- (2) Se ajustează valorile contoarelor.
- (3) Într-o a doua trecere, utilizând aceste contoare, se mută direct articolele în poziția lor ordonată în tabloul b .

```

enum {n = 100, m = 10};
typedef int tip_tablou[n];
int contor[m];
tip_tablou a,b;
int i,j;
int main(int argc, const char* argv[])
{ /*inițializare contoare*/
    for(j=0;j<m-1;j++)
        contor[j]= 0;
    /*determinarea distribuției cheilor*/
    for(i=0;i<n-1;i++)
        contor[a[i]]= contor[a[i]]+1;
    /*ajustare contoare*/
    for(j=1;j<m-1;j++)
        contor[j]= contor[j-1]+ contor[j];
    /*sortare cu determinarea distribuției cheilor în b*/
    for(i=n-1;i>0;i--)
    {
        b[contor[a[i]]-1]=a[i];
        contor[a[i]]= contor[a[i]]-1;
    }
    /*mută tabloul b în a*/
    for(i=0;i<n-1;i++)
        a[i]= b[i];
    return 0;
}
/*Sortare cu determinarea distribuției cheilor*/

```

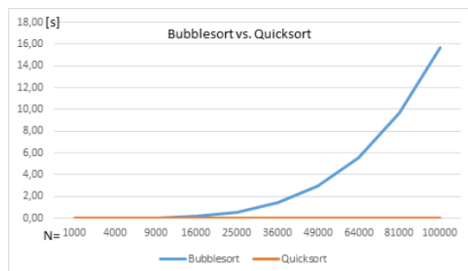
Lucrarea nr. 4-5. Tablouri – Tehnici avansate de sortare



2. Aplicabilitatea tehnicilor avansate de sortare

Aplicabilitate

Următoarea imagine arată performanța (exprimată în secunde) pentru implementările algoritmilor Bubblesort ($O(n^2)$) și Quicksort $O(n \log n)$ pe o anumită platformă pentru sortarea unor tablouri de numere aleatoare cu un număr de elemente cuprins între 1000 și 100000.



Numărul de elemente (N)	Timp Execuție Quicksort [s]	Timp Execuție Bubblesort [s]
1000	0,00	0,00
4000	0,00	0,00
9000	0,00	0,03
16000	0,00	0,17
25000	0,00	0,56
36000	0,00	1,41
49000	0,00	2,99
64000	0,01	5,60
81000	0,01	9,67
100000	0,01	15,64

Când este indicat să folosim algoritmi de sortare avansați?

Când numărul elementelor ale tabloului de sortat este mare, sau când dorim o performanță mai bună de timp.

3. Definiții si noțiuni teoretice

Noțiuni teoretice

În cele ce urmează vom presupune ca tablourile de sortat sunt definite după cum urmează:

```
typedef struct {
    int cheie;
    //... alte campuri;
} tip_element;
```

```
tip_element a[nr_max_elemente];
```

Se menționează că tipul cîmpului "cheie" poate fi orice tip pe care e definită o relație de ordonare (întreg, real, șir de caractere, enumerare).

După cum am văzut în laboratoarele anterioare, algoritmi de sortare se deosebesc între ei prin eficiență, timp de execuție necesar, exprimat prin funcția $O(f(n))$. Sunt folosiți pentru aprecierea eficienței și numărul de comparații efectuate asupra cheilor pentru sortare (notat cu C) și numărul de mișcări (atribuiri) de elemente (M).

4. Metode de sortare care nu iau în considerare structura și valorile cheilor

4.1. Sortarea prin inserție cu diminuarea incrementului (Shellsort)

Principiu de funcționare:

Algoritmul reprezintă o perfecționare a metodei de sortare prin inserție. Se alege o secvență de numere naturale $h_1, h_2, \dots, h_t$ numite incrementi, care satisfac condițiile $h_t = 1$ și $h_{i+1} < h_i$ pentru $i = 1, t-1$.

Se realizează t treceri asupra tabloului, la fiecare trecere i luându-se în considerare elementele $a[1]$, $a[1+h_i]$, $a[1+2 \cdot h_i]$ etc. Aceste elemente se sortează aplicând metoda inserției, cu utilizarea fanionului. S-a demonstrat că eficiența algoritmului crește dacă valorile incrementilor nu sunt puteri ale lui 2. Pentru a putea folosi tehnica fanionului la această metodă este necesar să se prelungească tabloul a spre stânga cu încă h_1 elemente.

Complexitatea: Algoritmul are complexitatea $O(n^2)$.

Observație: Algoritmul lucrează pe tablouri de lungime n , fiind de clasă $O(n^2)$, clasa ce caracterizează, în mod normal, algoritmi de sortare mai puțin performanți. Cu toate acestea, algoritmul este vizibil mai rapid decât algoritmi obișnuiți din clasa $O(n^2)$.

Implementarea algoritmului în C:

```
#define tt 4          //exemplu pentru tabloul h de dimensiune 4
void shellsort(tip_element a[], int n)
{
    tip_indice i, j, pas;
    tip_element temp;
    unsigned char m;
    int h[tt];

    /*ex. de atribuire directa incrementi pentru un tablou h de 4 elemente*/
    h[0] = 9;
    h[1] = 5;
    h[2] = 3;
    h[3] = 1;
    for (m = 0; m < tt; ++m)
    {
        pas = h[m];
        for (i = pas; i < n; ++i)
        {
            temp = a[i];
            for (j = i; j >= pas && a[j - pas].cheie > temp.cheie; j = j - pas)
            {
                a[j] = a[j - pas];
            }
            a[j] = temp;
        } /*for*/
    } /*for*/
} /*Shellsort*/
```

4. Metode de sortare care nu iau în considerare structura și valorile cheilor

4.2. Sortarea prin partitionare (Quicksort)

Principiu de funcționare:

Se alege un element $a[k]$ al tabloului, numit pivot. Pivotul poate fi ales primul element, ultimul element, elementul din mijloc sau oricare element ales aleator, în funcție de implementare.

Pe un sortare crescătoare, se parcurge tabloul de la stânga la dreapta, până când se găsește primul element $a[i].cheie > x.cheie$. Apoi, se parcurge tabloul de la dreapta, până când se găsește primul element $a[j].cheie < x.cheie$, apoi cele două elemente se interschimbă. Se repetă aceste operații până când s-a realizat o pargurare a întregului tablou ($i > j$). În acest moment pivotul se află pe poziția finală în tabloul sortat (în stânga sunt doar elemente mai mici sau egale cu pivotul, iar în dreapta sunt doar elemente mai mari sau egale cu pivotul).

Procedura descrisă anterior se aplică în continuare pe rând celor două partiții obținute (cele ce cuprinde elementele din stânga pivotului și cele ce cuprinde elementele din dreapta pivotului). Apoi fiecare partiție parcursă se va împărți la rândul ei în alte două și așa mai departe, până când fiecare partiție ajunge să fie formată dintr-un singur element.

Complexitatea algoritmului este $O(n \log n)$ în caz uzual, dar poate ajunge $O(n^2)$, în cazul cel mai defavorabil când pivotul este ales tot timpul elementul minim, sau respectiv elementul maxim.

Implementarea algoritmului în C:

Metoda Quicksort se poate implementa în două moduri: recursiv și nerecursiv. În ambele cazuri s-a convenit ca elementul x să fie ales la mijlocul tabloului (respectiv partiției). Procedura descrisă se aplică în continuare pe rând celor două partiții obținute, apoi celor patru partiții ș.a.m.d., până când fiecare partiție ajunge să fie formată dintr-un singur element.

În cazul algoritmului nerecursiv este necesară o stivă care memorează limitele uneia dintre cele două partiții care apar la o trecere, și anume limitele partiției care este tratată a doua (aici, partiția dreaptă).

Observație: Algoritmul Quicksort este cel mai rapid din clasa algoritmilor care nu țin cont de valoarea cheii. Dezavantajul său este că este un algoritm recursiv și solicită stiva sistemului sau o altă stivă (în varianta de implementare nerecursivă).

```
void quicksort(tip_element a[], int s, int d) {
    int i = s, j = d;
    tip_element x = a[(s + d) / 2];
    do {
        while (a[i].cheie < x.cheie) i++;
        while (a[j].cheie > x.cheie) j--;
        if (i <= j) {
            tip_element temp = a[i];
            a[i] = a[j];
            a[j] = temp;
            i++;
            j--;
        }
    } while (i <= j);

    if (s < j) quicksort(a, s, j);
    if (d > i) quicksort(a, i, d);
} /*quicksort*/
```

4. Metode de sortare care nu iau in considerare structura si valorile cheilor

4.3. Sortarea prin metoda ansamblelor (Heapsort)

Principiu de funcționare:

Se numește ansamblu (heap) o secvență de chei $h_0, h_2, \dots, h_{n-1}$ care satisfac condițiile:

$$h_i \leq h_{2i+1} \text{ și } h_i \leq h_{2i+2}, i=0, n/2.$$

Se aduce tabloul la forma unui ansamblu, adică pentru orice i, j, k din intervalul $[0, n-1]$, unde $j=2^i$ și $k=2^i+1$, să avem $a[j].cheie \leq a[i].cheie$ și $a[k].cheie \leq a[i].cheie$ (*).

Se observă că în acest caz $a[0]$ este elementul cu cheia minimă în tablou. Se interschimbă elementele $a[0]$ și $a[n-1]$ și se aduce subtabloul $a[0], \dots, a[n-2]$ la forma de ansamblu, apoi se interschimbă elementele $a[0]$ și $a[n-2]$ și se aduce subtabloul $a[0], \dots, a[n-3]$ la forma de ansamblu ș.a.m.d. În final rezultă tabloul ordonat invers. Dacă se schimbă sensul relațiilor în condițiile (*), atunci se obține o ordonare directă a tabloului ($a[0]$ va fi elementul cu cheia maximă).

Aducerea unui tablou la formă de ansamblu se bazează pe faptul că subtabloul $a[n-1/2+1], \dots, a[n-1]$ este deja un ansamblu. Acest subtablou se va extinde mereu spre stânga cu câte un element al tabloului, până când se ajunge la $a[0]$. Elementul adăugat va fi glisat astfel încât subtabloul extins să devină ansamblu.

Complexitatea este $O(n \log n)$.

Observație: Această metodă este de aceeași clasă de complexitate cu Quicksort, dar este mai lentă decât aceasta. Heapsort prezintă avantajul că nu este o metodă recursivă și deci nu solicită stiva sistemului sau o altă stivă așa cum face Quicksort.

Implementarea algoritmului în C:

Implementarea algoritmului în C:

```
void deplasare(int *s, tip_element *temp, int *d, tip_element a[])
//realizeaza glisarea elementului a[s] in pozitia corecta
{
    int i, j;
    int ret = 0;
    i = *s; j = 2 * i; *temp = a[i]; ret = 0;
    while ((j <= *d) && (!ret))
    {
        if (j < *d && a[j].cheie < a[j + 1].cheie)
            j = j + 1;

        if ((*temp).cheie < a[j].cheie)
        {
            a[i] = a[j]; i = j; j = 2 * i;
        }
        else
            ret = 1;
    } /*while*/
    a[i] = *temp;
} /*Deplasare*/
```

```
void heapsort(tip_element a[], int n)
{
    int s, d;
    tip_element temp;
    /*construcție ansamblu*/
    s = (n / 2) + 1; d = n - 1;
    while (s > 0)
    {
        s = s - 1; deplasare(&s, &temp, &d, a);
    } /*while*/
    while (d > 0) /*sortare ansamblu*/
    {
        temp = a[0]; a[0] = a[d]; a[d] = temp;
        d = d - 1; deplasare(&s, &temp, &d, a);
    }
} /*Heapsort*/
```


5. Metode de sortare care tin cont de valorile cheilor

5.1. Tehnica binsort

Se poate aplica în cazul în care cheile sunt valori de tip întreg cuprinse în intervalul 0..N-1 și nu există duplicate.

Principiu de funcționare: se parcurge tabloul a, verificându-se dacă elementul a[i] are cheia j egala cu i. Dacă j < i se interschimba elementele a[i] și a[j].

Complexitatea algoritmului este O(n).

Observație: Aparține celei mai bune clase de eficiență, dar nu poate fi folosit pentru sortarea oricărui tip de tablou.

Implementarea algoritmului în C:

```
void BinSort(tip_element a[], int n)
{
    int i;
    tip_element temp;
    for (i = 0; i < n; i++)
    {
        while (a[i].cheie != i)
        {
            temp = a[i];
            a[i] = a[temp.cheie];
            a[temp.cheie] = temp;
        }
    }
} //BinSort
```

5. Metode de sortare care tin cont de valorile cheilor

5.2. Metode de sortare care folosesc baze de numeratie (RadixSort)

Aceste metode tratează cheile de sortat ca numere reprezentate într-o anumita baza de numerație R (radix) și lucrează cu cifrele individuale ale numărului. Pentru implementarea pe sisteme de calcul se pretează metodele care utilizează R=2 și care lucrează deci cu biți ce formează numerele.

În sortarea radix cu R=2 operația fundamentală necesară este extracția unui set contiguu de biți dintr-un număr. Ca atare se va defini o funcție care va returna "j biți care apar la k poziții de la marginea dreapta a lui x", unde x este numărul considerat:

```
unsigned biti(unsigned x, int k, int j)
{
    return (x >> k) & ~(~0 << j);
}
```

5. Metode de sortare care tin cont de valorile cheilor

5.3. Sortarea radix prin interschimbare

Principiu de funcționare:

Algoritmul se bazează pe faptul că rezultatul comparației a două chei este determinat numai de valoarea biților din prima poziție la care ele diferă (considerând biții de la stânga spre dreapta). Astfel, cheile care au primul bit=0 sunt trecute în fața celor care au primul bit=1; în continuare în fiecare grup se aplică aceeași metodă, pentru bitul următor ș.a.m.d. Procesul se desfășoară ca și la metoda Quicksort: se baleiază tabloul de la stânga spre dreapta până se găsește o cheie care începe cu 1; se baleiază apoi de la dreapta spre stânga până se găsește o cheie care începe cu 0; se interschimba cele două chei; se continuă în felul acesta până când indicii de parcurgere se întâlnesc: în acest moment tabloul are două partiții. Se reia procedura expusă, considerând următorul bit, pentru fiecare din partițiile rezultate etc.

Complexitatea este O(kn), unde k este numărul de biți pe care este reprezentat numărul.

Observație: Pentru o distribuție normală a bitilor metoda lucrează mai repede chiar și decât Quicksort.

Implementarea algoritmului în C:

Implementarea algoritmului în C:

```
void radinters (int stanga,int dreapta, int b, tip_element a[])
/*stanga, dreapta - limitele curente ale tabloului de sortat*/
/*b - lungimea în biți a cheii de sortat*/
{
    int i,j;
    int t_;
    if ((dreapta>stanga) && (b>=0))
    {
        i = stanga; j = dreapta;
        do {
            while((biti(a[i].cheie,b,1)==0)&&(i<j)) i= i+1;
            while((biti(a[j].cheie,b,1)==1)&&(i<j)) j= j-1;
            t_ = a[i]; a[i]= a[j]; a[j]= t_;
        } while (! (j==i));
        if (biti(a[dreapta].cheie,b,1)== 0)
            j= j+1; /* dacă ultimul bit testat este 0 se reface lungimea partiției */
        radinters(stanga,j-1,b-1);
        radinters(j,dreapta,b-1);
    }
} /*RadixInterschimb*/
```

5. Metode de sortare care tin cont de valorile cheilor

5.4. Sortarea radix directa

Principiu de funcționare:

Metoda examinează toți biții care formează cheia de la dreapta la stânga, considerând cite "m" biți deodată (unde "m" este un divisor al lungimii cheii - "b"). În felul acesta sortarea se execută în b/m treceri. Aplicarea acestei metode de sortare presupune folosirea unui tablou auxiliar T care memorează rezultatul fiecărei treceri, precum și a unui tablou Numar care sa conțină distribuția cheilor în raport cu cei "m" biți considerați la un moment dat: adică pentru fiecare combinație posibilă de "m" biți trebuie sa se determine numărul cheilor care conțin acea combinație; de aici rezulta ca dimensiunea tabloului Numar este 2^m . Pentru ca algoritmul sa funcționeze corect este necesar ca fiecare sortare după "m" biți sa fie stabilă.

Complexitatea este $O(kn)$.

Observație: Se observă ca numărul de treceri se reduce dacă "m" este mare, ceea ce conduce însă la creșterea dimensiunii tabloului Numar. S-a constatat ca metoda este eficientă pentru m=4 sau m=8.

Implementarea algoritmului în C:

```
/*RadixInterschimb*/
#define m 2
#define m1 m*m
tip_element t[m1];
void radixdirect(tip_element a[], int n, int b)
// b-numarul de biti pe care este reprezentata cheia
{
    int i,j,trecere;
    int numar[m1]; /*m1:=2^m*/
    int aux;
    for( trecere= 0; trecere<=(b/m)-1; trecere++)
    {
        for( j= 0; j <= m1-1; j++)
            numar[j]= 0;
        for( i= 0; i <= n-1; i++)
        {
            aux=biti(a[i].cheie,trecere*m,m);
            numar[aux]= numar[aux]+1;
        }
        for( j= 1; j <= m1-1; j++) /* { * } */
            numar[j]= numar[j-1]+numar[j];
        for( i= n-1; i >= 0; i--)
        {
            aux=biti(a[i].cheie,trecere*m,m);
            t[numar[aux]-1]= a[i]; /* { ** } */
            numar[aux]= numar[aux]-1;
        }
        for( i= 0; i < n; i++) a[i]= t[i];
    }
} /* radixdirect*/
```